CSS assignments

Task 1: Grid vs Flexbox (Layout Systems)

Requirements:

- Research **CSS Grid** and **Flexbox**
- Explain:
    - What is a Grid?

      Ans) Grid is a layout system that allows users to arrange elements in rows and columns. Grid allows developers to easily create complex web layouts.
      Users can choose how to size rows and columns tracks or they can react to the size of the content. Direct children of the grid container will be automatically placed onto this grid or, you can place the items in the precise location that you want.

    - What is Flexbox?

      Ans) Flexbox is a layout system that is used to arrange items inside a container either horizontally or vertically. It allows elements to be positioned in a flexible and responsive way, making it easier to control alignment, spacing, and distribution of space within a layout.

    - Key differences between them

      Ans) Flexbox is used for a one-dimensional layout, with rows OR columns whereas Grid is used for a two-dimensional layout, with rows AND columns.

- Identify:
    - When to use Grid

      Grid is used when you need to design a full-page layout or structure content in both rows and columns at the same time. It is good for complex layouts (like dashboards) where precise control over both dimensions is needed.

    - When to use Flexbox

      Flexbox is used when you need to arrange items in a single direction (row or column). It is better used to align smaller components like navigation bars, button or cards in a simple and flexible way.

Task 2: Grid (Layout Systems)

Code:

- Create:
  - One layout using **Grid** CSS Grid Assessment

Live Page: Link

Task 3: Flexbox (Layout Systems)

Code:

- Create:
  - One layout using **Flexbox** FlexBox Assessment

Live Page: Link

Task 4: Float and Clear (Float Reset)

Requirements:

- Research:
  - What is float?

Ans) Float is a CSS property used to position an element to the left or right of its container, allowing other content to wrap around it.
The float property can have one of the following values:
left - The element floats to the left of its container
right - The element floats to the right of its container
none - Default. The element does not float and is displayed just where it occurs in the text
inherit - The element inherits the float value of its parent

- How can we clear floats?

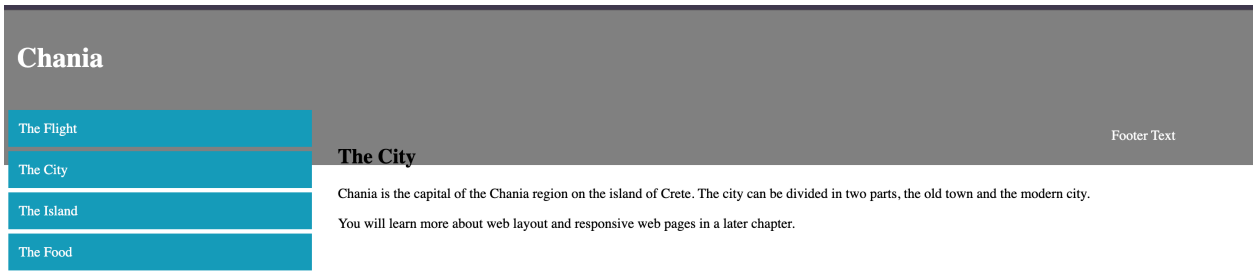
Ans) When elements are floated, they are removed from normal document flow, which can cause layout issues.
To fix this, we use “clear” property which says what should happen with the element that is next to a floating element. This property prevents elements from wrapping around or beside the floated content.

Clear :left clears left floats
clear: right clears right floats
clear: both clears both sides

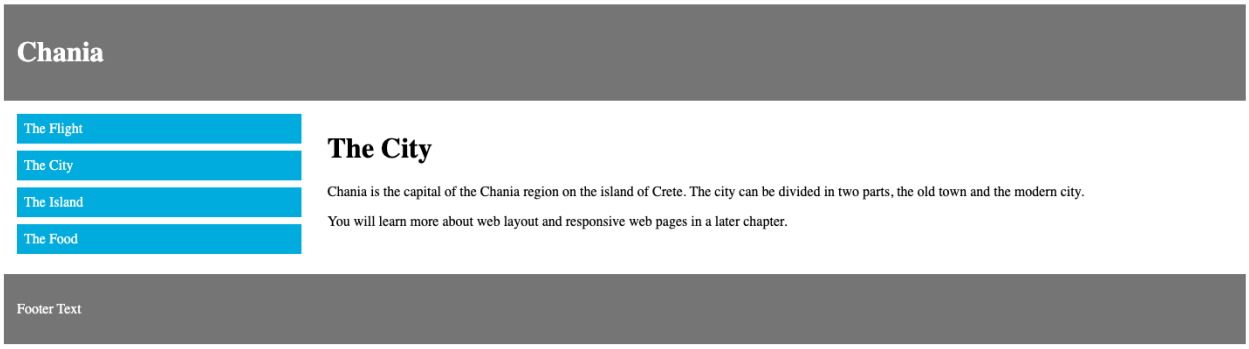
- What problems does it create?

Ans) Float can cause layout issues because floated elements are removed from the normal document flow, which can make parent containers collapse. It can also lead to overlapping elements, unpredictable spacing, and extra clearing work is. needed. Because of these issues, float is difficult for complex layouts.

Example of layout problem created by flex:



Code:



- Create a layout using float
  - The left menu column and right content should be placed with float and width. (Do not use grid or flex)
- Show how to reset the float. List multiple ways you can do it.

Ans) To reset (clear) float:

1. Using clear:

```
.clear {
    clear: both;
}
```

2. Clearfix using ::after

```
.clearfix::after {
  content: "";
  clear: both;
  display: table;
}
```

the ::after pseudo-element is used to ensure that the parent container properly encloses its floated child elements:

3. Using overflow:(auto or hidden) on parent

Live Page: Link

Task 5: Display vs Visibility

Topics to Research:

- display: block
- display: none
- opacity: 0
- visibility: hidden

Requirements:

Explain the differences:

- Layout behavior
- Space occupation
- Interaction (clickable or not)
- Rendering differences

Ans)

display: block is the standard state for most layout elements, where object takes up its full width

display: none removes the element from the layout logic. The surrounding elements will shift to fill the gap and because the element is no longer exists, it is impossible to click on or interact with in any way.

visibility: hidden make the element invisible, but it still takes up space in the layout, so the page does not shift and a blank area is left where it was. Even though it is still part of the layout, it cannot be clicked, hovered, or focused, and it is not accessible.

opacity: 0 only changes how visible an element is. The element is still present on the page, it can still be interacted with and it takes up space. For example, even if a button has opacity: 0, it can still be clicked or focused using the keyboard, even though it cannot be seen. Unlike display: none, it remains in the layout, and unlike visibility: hidden, it still allows interaction. It also supports transitions and animations.

Code:

- Create 3 elements and apply:
  - display: none
  - visibility: hidden
  - opacity: 0

- Show how each behaves differently
  When display: none was applied, I observed that the entire content got removed completely whereas visibility: hidden only made it invisible but the space remains. Opacity only made it transparent and was still interactive.

- Transition differences between opacity and display block none

  When opacity changes from 0 to 1, the element gradually fades in or out because it is still part of the layout even when invisible. However, display: none completely removes the element from the page, so it takes up no space and cannot be interacted with. Because of this, it cannot be hovered over, and user needs to hover over a parent container to make it appear.

  Live Page: Link

Task 6: CSS Selectors & Pseudo-Classes

Pseudo-classes to explore:

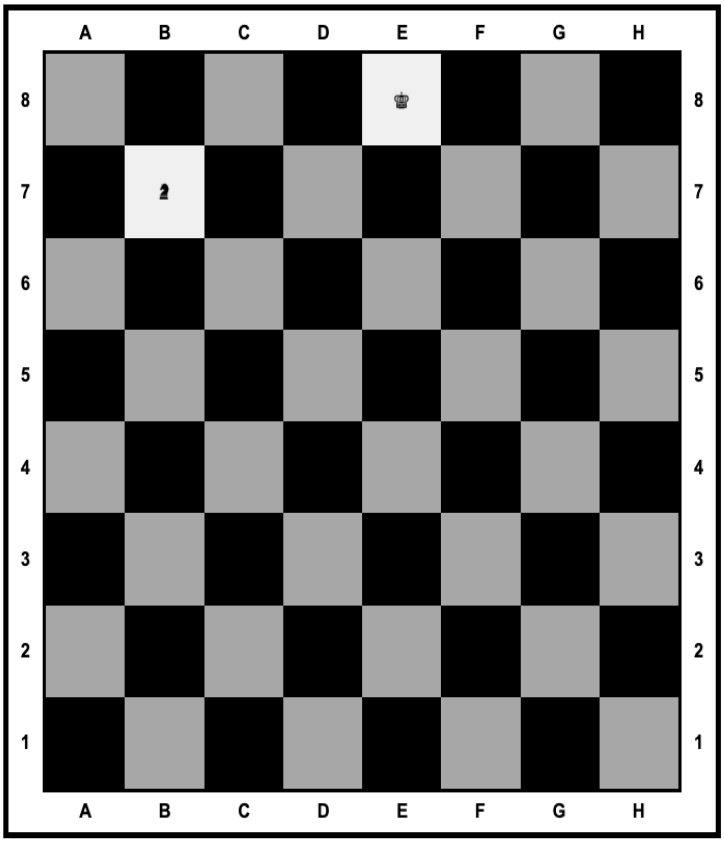
- :hover
- :active
- :focus
- :nth-child()
- :first-child
- :last-child
- :is()
- :where()
- :not()

Code:

- Create a demo page where:
  - Buttons change style on hover
  - Input field changes on focus

Live Page: Link

Task 7: CSS Selectors & Pseudo-Classes

- Create a chessboard page example with above pseudo classes
  - Hover for the box should change the color
  - The box containing image of chess pieces should be brighter other should be dimmed
- 

Live Page: Link

Task 8 : Animation

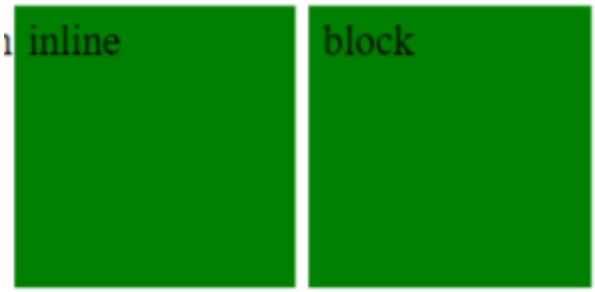
Recreate the gif below using css animations.

Live Page : Link

Task 9 : Display property

How can you remove white spacing between elements with display: inline-block ? Explain why there was spacing in the first place.

Make the layout from below with both columns will have a display: inline-block. Research and show various methods.



The spacing happens because inline-block elements are treated like inline text by the browser. When you write HTML like this:

```
<div class="box">
</div>
<div class="box">
</div>
```

There is a newline (whitespace) between the two elements. The browser interprets that like a space between words in text.

1) One solution is is to remove the whitespace during coding. For example:
   Writing:
   Instead of:

2) by using **display:flex** so that the child is treated as a flex item. Since they are no longer part of the inline text flow, whitespace does not create gaps between elements.

3) Implementing negative margin: though not recommended

Live Page: Link